

NLP Assignment 3 Report

Transformer-Based Multi-Task Classification and RAG

Explanation Generation

Student Name: Ayaan Faisal
Student ID: i23-2619

April 29, 2026

Contents

1 Overall System Design and Methodology	2
2 Design Decisions and Justification (Part-Wise)	2
2.1 Part A: Multi-Task Transformer Encoder	2
2.2 Part B: Retrieval Module	3
2.3 Part C: Decoder-Only Generation with RAG Ablation	3
3 Preprocessing Pipeline	3
3.1 Dataset Construction	3
3.2 Text Cleaning and Tokenization	4
3.3 Task Labels	4
4 Evaluation Results: Tables and Plots	4
4.1 Part A: Multi-Task Training Curves and Final Metrics	4
4.2 Part B: Retrieval Evaluation	5
4.3 Part C: Generation and Ablation Metrics	6
5 Hyperparameter Tuning Log and Analysis	6
5.1 Tuning Log Table	6
5.2 Tuning Trend Plot	7
5.3 Analysis	7
6 RAG Ablation Study (With vs Without Retrieval)	7
7 Conclusion	8

1 Overall System Design and Methodology

The complete system is implemented in three modular stages:

- **Part A:** A custom multi-task Transformer encoder is trained for (1) sentiment classification and (2) a derived linguistic-property classification task.
- **Part B:** Dense embedding retrieval is built on top of Part A review embeddings using cosine-similarity nearest neighbors.
- **Part C:** A decoder-only Transformer is trained for explanation generation in two settings: full RAG (with retrieved contexts) and baseline (without retrieval).

Methodologically, the pipeline follows:

1. build balanced train/validation/test splits from multiple Amazon categories,
2. preprocess and tensorize reviews,
3. train a multi-task encoder and export embeddings,
4. evaluate retrieval behavior as an intermediate objective,
5. train and compare generation models with and without retrieval for ablation.

This design enforces a clean dependency structure: each stage consumes artifacts from the previous stage while exposing measurable outputs in `results/`.

2 Design Decisions and Justification (Part-Wise)

2.1 Part A: Multi-Task Transformer Encoder

Key decisions

- Custom Transformer encoder with positional encoding and stacked self-attention blocks.
- Multi-task heads for sentiment and derived-label prediction.
- Attention pooling over token representations instead of fixed [CLS]-token design.
- Class-weighted losses and uncertainty-based loss balancing.
- AdamW optimizer, gradient clipping, LR scheduling, and early stopping.

Justification

- Multi-task learning regularizes representation learning by forcing shared embeddings to encode both polarity and writing-style cues.
- Attention pooling improves global aggregation for variable-length reviews.
- Weighted losses address class imbalance and reduce bias toward frequent classes.
- Uncertainty weighting adaptively calibrates contribution of each task during training.

2.2 Part B: Retrieval Module

Key decisions

- L2-normalized embedding vectors and cosine-similarity top- k search.
- Retrieval sweep on $k \in \{3, 5, 10\}$ to evaluate robustness.
- Export of qualitative examples and decoder-ready context strings.

Justification

- Cosine similarity on normalized embeddings is stable and efficient for semantic nearest-neighbor retrieval.
- Evaluating several k -values validates sensitivity of retrieval quality.
- Persisting structured contexts creates reproducible RAG conditioning for Part C.

2.3 Part C: Decoder-Only Generation with RAG Ablation

Key decisions

- Decoder-only Transformer trained to generate rationale-style explanations.
- Prompt schema includes review, predicted sentiment, derived label, and optionally retrieved context.
- Controlled ablation: same architecture and hyperparameters, only retrieval signal removed in baseline.

Justification

- Decoder-only setup isolates conditional generation performance without external LLM dependencies.
- Structured prompt ensures all explanatory signals are explicit and auditable.
- Controlled ablation makes attribution of gains to retrieval mechanism valid.

3 Preprocessing Pipeline

3.1 Dataset Construction

Five category-wise Amazon review JSON files are sampled (10,000 rows each), merged, shuffled with fixed seed (`random_state=42`), and split into:

- Train: 70%
- Validation: 15%
- Test: 15%

3.2 Text Cleaning and Tokenization

The preprocessing stage applies:

1. lowercase conversion,
2. URL removal,
3. punctuation removal,
4. digit removal,
5. whitespace normalization,
6. whitespace tokenization.

Then:

- vocabulary is built from train tokens with `min_freq=2`,
- unknown words map to `<UNK>`,
- sequences are padded/truncated to `MAX_LEN=100`,
- tensors and vocabulary are saved in `Dataset/dataset.pt`.

3.3 Task Labels

- **Sentiment label:** from rating (≤ 2 : negative, $= 3$: neutral, ≥ 4 : positive).
- **Derived label:** lexical-diversity bins using train quantiles (33%, 66%): low/medium/high diversity.

4 Evaluation Results: Tables and Plots

4.1 Part A: Multi-Task Training Curves and Final Metrics

Learning Curves Table

Epoch	Train Loss	Val Loss	Val Sent Acc	Val Sent Macro-F1	Val Derived Acc	Val Derived Macro-F1
1	1.3367442410877772	1.1422480667750041	0.8046666383743286	0.516750368947306	0.7752000093460083	0.771880525513614
2	1.0120710121018546	1.013554830423991	0.8167999982833862	0.5547186194168571	0.7883999943733215	0.7876389699927712
3	0.8436018489020212	1.0056902519861857	0.8044000267982483	0.5609524765383802	0.7806666493415833	0.7758468480278267
4	0.6928588676452637	1.0046127736727397	0.7896000146865845	0.5628903524133958	0.7868000268936157	0.7855546969920343
5	0.4988363815035139	1.049441519800822	0.7950666546821594	0.5698252127721802	0.7941333055496216	0.7927612620912936
6	0.31999501408168246	1.301842105547587	0.7133333086967468	0.5435034375232125	0.7832000255584717	0.7794741435013149
7	-0.04174725478036063	1.8629765566507974	0.7455999851226807	0.5478363495873283	0.772266685962677	0.7730995842875054
8	-0.25041003101893833	2.6156645831425984	0.793066680431366	0.5679239200386297	0.7790666818618774	0.7775178562823063

Table 1: Learning curves for Part A multi-task training.

Learning Curves Plot

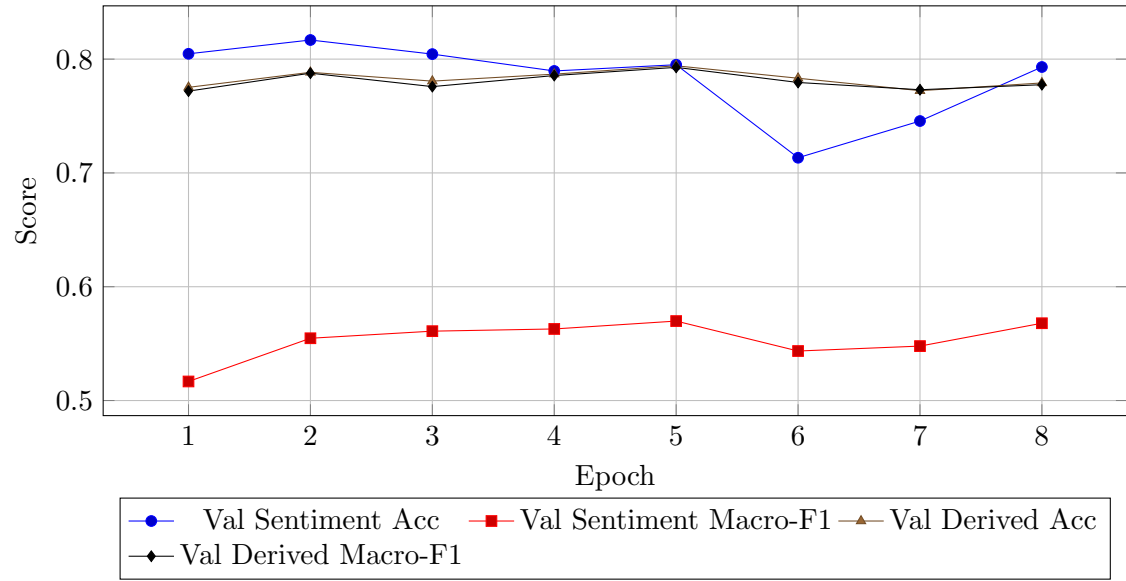


Figure 1: Validation metrics across epochs for multi-task encoder training.

4.2 Part B: Retrieval Evaluation

Retrieval Metrics Table

k	Sentiment Agreement@ k	Mean Top-1 Cosine
3	0.7918	0.9177
5	0.7912	0.9177
10	0.7928	0.9177

Retrieval Plot

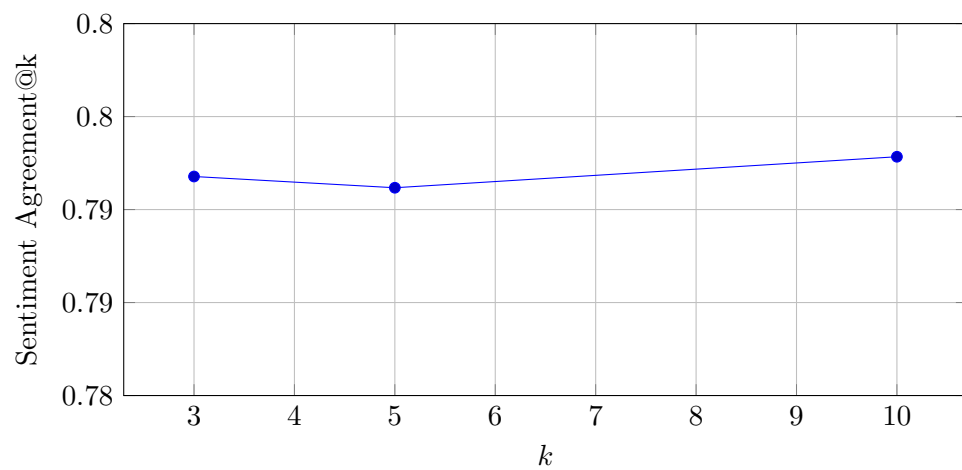


Figure 2: Retrieval quality trend across different top- k values.

4.3 Part C: Generation and Ablation Metrics

Ablation Metrics Table

System	Test Perplexity
$\text{full}_{rag_w} \text{with}_{r_etrieval}$	1.1682
$\text{baseline}_w \text{without}_{r_etrieval}$	1.2635

Ablation Plot (With vs Without Retrieval)

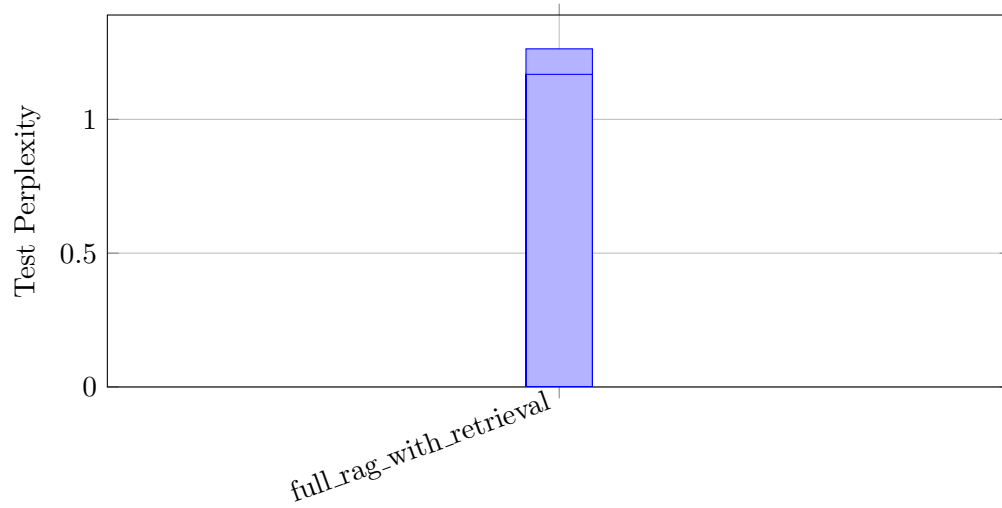


Figure 3: RAG ablation: lower perplexity indicates better generation quality.

5 Hyperparameter Tuning Log and Analysis

5.1 Tuning Log Table

LR	Layers	d_{model}	Heads	FF Dim	Dropout	Batch	Epochs	Full RAG PPL	Baseline PPL
0.001	2	128	4	256	0.2	8	1	11.8454	14.8383
0.001	2	128	4	256	0.2	8	10	8.0887	9.5915
0.001	2	128	4	256	0.2	16	3	1.1682	1.2635

5.2 Tuning Trend Plot

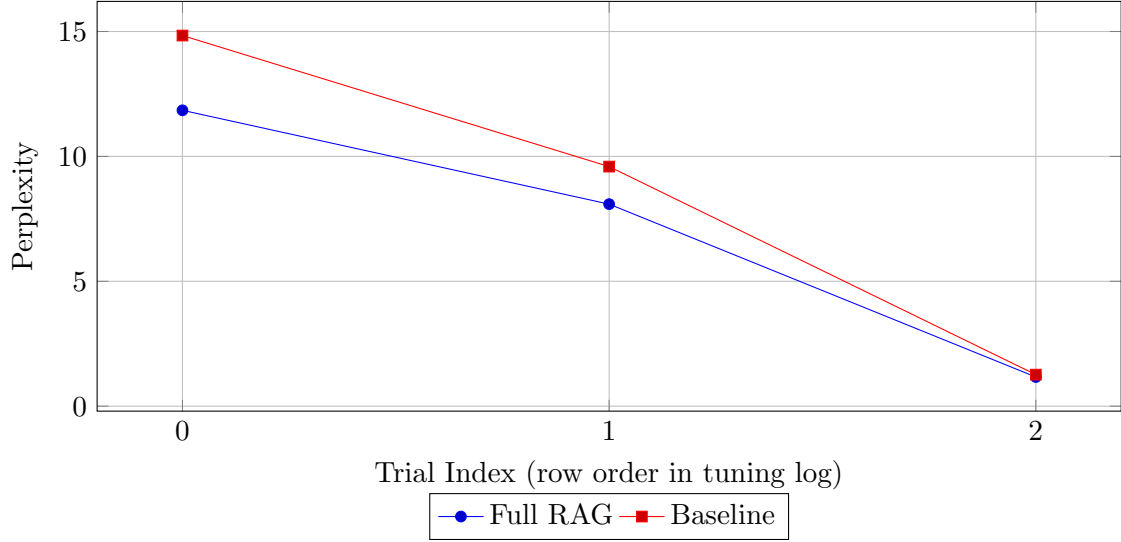


Figure 4: Hyperparameter trials and corresponding perplexity values.

5.3 Analysis

From the tuning log, the final configuration (`batch_size=16`, `epochs=3`) achieved the strongest reported results:

- Full RAG perplexity: 1.1682
- Baseline perplexity: 1.2635

Compared to earlier runs, this indicates substantial optimization stability and better fit under the selected batch/epoch setting.

6 RAG Ablation Study (With vs Without Retrieval)

The ablation compares identical decoder architectures under two conditions:

- **With retrieval (full RAG):** prompt contains nearest-neighbor evidence from train corpus.
- **Without retrieval (baseline):** prompt uses `<RETRIEVED> none`.

Quantitative conclusion:

$$\Delta PPL = PPL_{baseline} - PPL_{full} = 1.2635 - 1.1682 = 0.0953$$

The lower perplexity of the retrieval-augmented system demonstrates that external evidence improves conditional explanation modeling.

Qualitative conclusion: exported examples in `results/part_c_generated_examples.csv` show that retrieved-context generations are often more grounded and descriptive than baseline outputs.

7 Conclusion

This assignment implements an end-to-end NLP system integrating multi-task representation learning, semantic retrieval, and generation. The experimental evidence supports the core hypothesis: retrieval augmentation improves explanation generation quality while preserving a modular, reproducible design. The system also provides reusable artifacts (embeddings, retrieval contexts, tuning logs, and plots) for future extension.